浙江大學
ZHEJIANG UNIVERSITY



Signal and System
Lab Report

Experiment Name	Lab04
Experiment Location	西 2-310
Name	Zhixuan Zheng
Student ID	3230103913
Experiment Date	June 3, 2025
Instructor	Xiaoming Chen

Contents

1 Problem1 a-c **1**

1.1 Problem . . . 1

1.2 Answer . . . 1

1.2.1 Answer(a) . . . 1

1.2.2 Answer(b) . . . 2

1.2.3 Answer(c) . . . 3

2 Problem1 d-e **4**

2.1 Problem . . . 4

2.2 Answer . . . 5

2.2.1 Answer(d) . . . 5

2.2.2 Answer(e) . . . 6

3 Problem2 a-c **7**

3.1 Problem . . . 7

3.2 Answer . . . 7

3.2.1 Answer(a) . . . 7

3.2.2 Answer(b) . . . 8

3.2.3 Answer(c) . . . 10

4 Problem2 d-h **12**

4.1 Problem . . . 12

4.2 Answer . . . 12

4.2.1 Answer(d) . . . 12

4.2.2 Answer(e) . . . 13

4.2.3 Answer(f) . . . 14

4.2.4 Answer(g) . . . 14

4.2.5 Answer(h) . . . 16

5 Problem3 a-c **17**

5.1 Problem . . . 17

5.2 Answer . . . 17

5.2.1 Answer(a) . . . 17

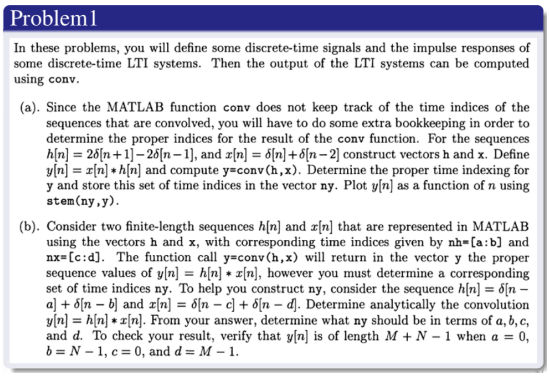
5.2.2 Answer(b) 18
5.2.3 Answer(c) 18

6 Problem3 d-f **19**
6.1 Problem 19
6.2 Answer 19
6.2.1 Answer(d) 19
6.2.2 Answer(e) 20
6.2.3 Answer(f) 22

7 Problem3 g-h **22**
7.1 Problem 22
7.2 Answer 22
7.2.1 Answer(g) 22
7.2.2 Answer(h) 23

1 Problem1 a-c

1.1 Problem



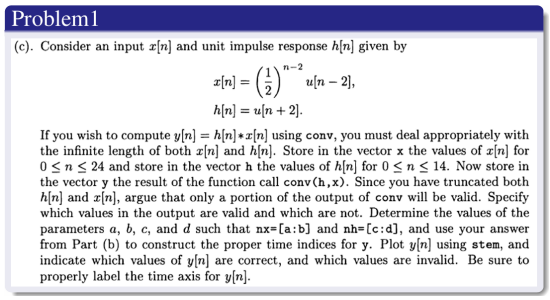
Problem1

In these problems, you will define some discrete-time signals and the impulse responses of some discrete-time LTI systems. Then the output of the LTI systems can be computed using `conv`.

(a). Since the MATLAB function `conv` does not keep track of the time indices of the sequences that are convolved, you will have to do some extra bookkeeping in order to determine the proper indices for the result of the `conv` function. For the sequences $h[n] = 2\delta[n+1] - 2\delta[n-1]$, and $x[n] = \delta[n] + \delta[n-2]$ construct vectors `h` and `x`. Define $y[n] = x[n] * h[n]$ and compute `y=conv(h,x)`. Determine the proper time indexing for `y` and store this set of time indices in the vector `ny`. Plot $y[n]$ as a function of n using `stem(ny,y)`.

(b). Consider two finite-length sequences $h[n]$ and $x[n]$ that are represented in MATLAB using the vectors `h` and `x`, with corresponding time indices given by `nh=[a:b]` and `nx=[c:d]`. The function call `y=conv(h,x)` will return in the vector `y` the proper sequence values of $y[n] = h[n] * x[n]$, however you must determine a corresponding set of time indices `ny`. To help you construct `ny`, consider the sequence $h[n] = \delta[n-a] + \delta[n-b]$ and $x[n] = \delta[n-c] + \delta[n-d]$. Determine analytically the convolution $y[n] = h[n] * x[n]$. From your answer, determine what `ny` should be in terms of a, b, c, and d. To check your result, verify that $y[n]$ is of length $M + N - 1$ when $a = 0$, $b = N - 1$, $c = 0$, and $d = M - 1$.

Figure 1: Problem1 a-c



Problem1

(c). Consider an input $x[n]$ and unit impulse response $h[n]$ given by

$$x[n] = \left(\frac{1}{2}\right)^{n-2} u[n-2],$$

$$h[n] = u[n+2].$$

If you wish to compute $y[n] = h[n] * x[n]$ using `conv`, you must deal appropriately with the infinite length of both $x[n]$ and $h[n]$. Store in the vector `x` the values of $x[n]$ for $0 \le n \le 24$ and store in the vector `h` the values of $h[n]$ for $0 \le n \le 14$. Now store in the vector `y` the result of the function call `conv(h,x)`. Since you have truncated both $h[n]$ and $x[n]$, argue that only a portion of the output of `conv` will be valid. Specify which values in the output are valid and which are not. Determine the values of the parameters a, b, c, and d such that `nx=[a:b]` and `nh=[c:d]`, and use your answer from Part (b) to construct the proper time indices for `y`. Plot $y[n]$ using `stem`, and indicate which values of $y[n]$ are correct, and which values are invalid. Be sure to properly label the time axis for $y[n]$.

Figure 2: Problem1 a-c

1.2 Answer

1.2.1 Answer(a)

For discrete-time signal convolution, the operation is defined as:

$$y[n] = h[n] * x[n] = \sum_{k=-\infty}^{\infty} h[k]x[n-k] \tag{1}$$

Where $h[n] = [2, 0, -2]$ corresponds to $n \in [-1, 0, 1]$, and $x[n] = [1, 0, 1]$ corresponds to $n \in [0, 1, 2]$.

The length of the convolution result is $(N_h + N_x - 1) = (3 + 3 - 1) = 5$, and the output sequence index range is $n \in [-1 + 0, 1 + 2] = [-1, 3]$.

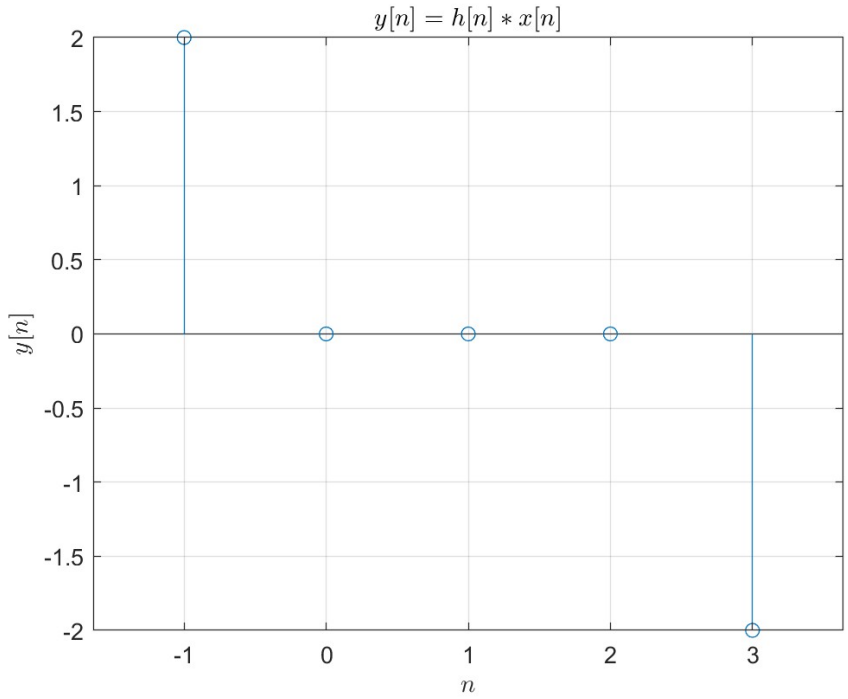


Figure 3: Problem 1(a): Convolution operation result

1.2.2 Answer(b)

For more complex signals, convolution follows the same principles. If $h[n]$ is non-zero in the range $n \in [a, b]$ and $x[n]$ is non-zero in the range $n \in [c, d]$, then the convolution result $y[n]$ is non-zero in the range $n \in [a + c, b + d]$.

When $a = 0$, $b = N - 1$, $c = 0$, $d = M - 1$, $n_y = [0 : M + N - 2]$ which is coincident with the expected result.

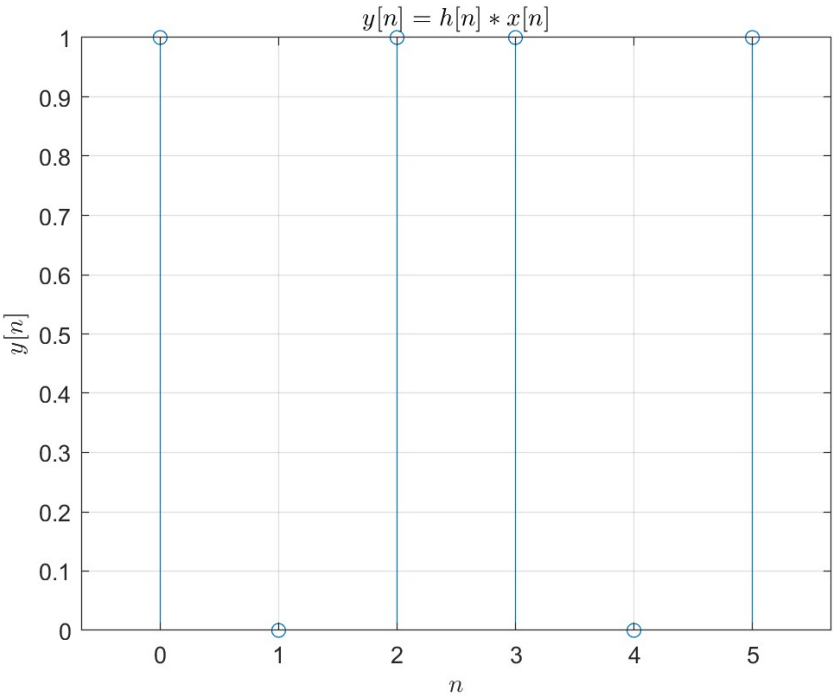


Figure 4: Problem 1(b): Finite-length sequence convolution

1.2.3 Answer(c)

For convolution of long sequences, the computational complexity is $O(N \cdot M)$, where N and M are the lengths of the two sequences respectively. The commutative, associative, and distributive properties of convolution also hold in discrete-time systems:

$$h[n] * x[n] = x[n] * h[n] \tag{2}$$

$$(h_1[n] * h_2[n]) * x[n] = h_1[n] * (h_2[n] * x[n]) \tag{3}$$

$$h[n] * (x_1[n] + x_2[n]) = h[n] * x_1[n] + h[n] * x_2[n] \tag{4}$$

From the experimental results, we can observe that $y'[n]$ is obtained by shifting $y[n]$ to the right by two time units within the range of $2 \leq n \leq 16$. The blue curve represents the difference between $y[n]$ and $y'[n]$. The value of $y'[n]$ is nearly correct when $8 \leq n \leq 16$, demonstrating the validity of convolution properties in practice.

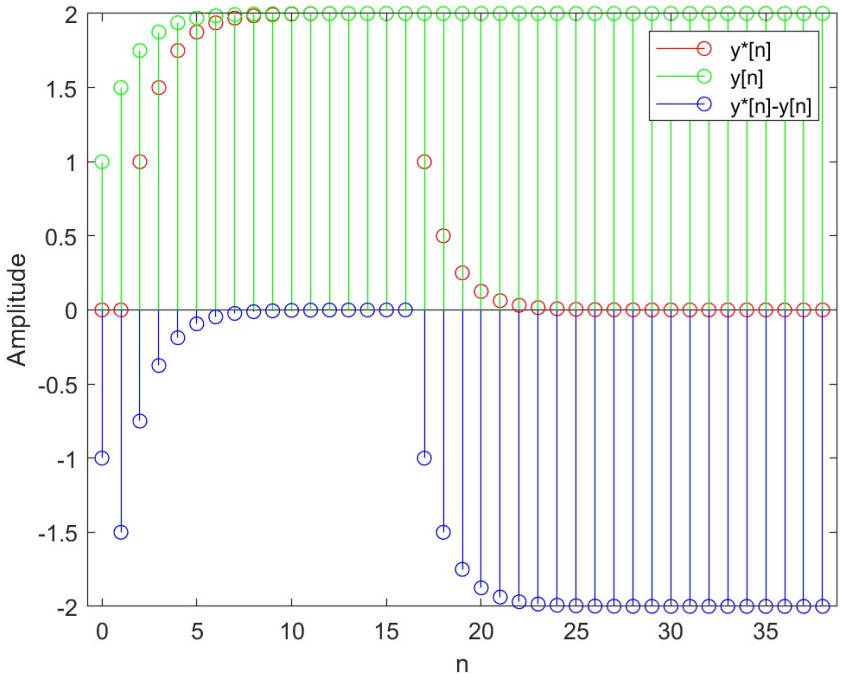
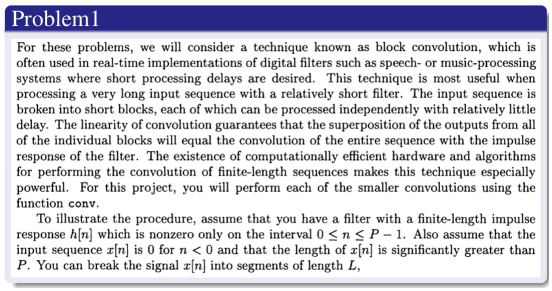


Figure 5: Problem 1(c): Verification of convolution properties

2 Problem1 d-e

2.1 Problem

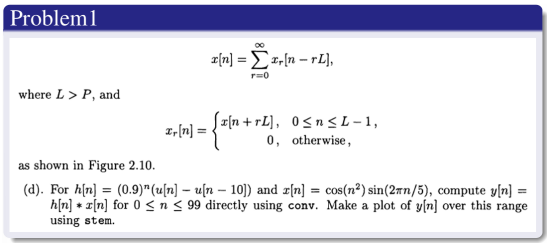


Problem1

For these problems, we will consider a technique known as block convolution, which is often used in real-time implementations of digital filters such as speech- or music-processing systems where short processing delays are desired. This technique is most useful when processing a very long input sequence with a relatively short filter. The input sequence is broken into short blocks, each of which can be processed independently with relatively little delay. The linearity of convolution guarantees that the superposition of the outputs from all of the individual blocks will equal the convolution of the entire sequence with the impulse response of the filter. The existence of computationally efficient hardware and algorithms for performing the convolution of finite-length sequences makes this technique especially powerful. For this project, you will perform each of the smaller convolutions using the function `conv`.

To illustrate the procedure, assume that you have a filter with a finite-length impulse response $h[n]$ which is nonzero only on the interval $0 \le n \le P-1$. Also assume that the input sequence $x[n]$ is 0 for $n < 0$ and that the length of $x[n]$ is significantly greater than P. You can break the signal $x[n]$ into segments of length L,

Figure 6: Problem1 d-e



Problem1

$$x[n] = \sum_{r=0}^{\infty} x_r[n - rL],$$

where $L > P$, and

$$x_r[n] = \begin{cases} x[n + rL], & 0 \le n \le L-1, \\ 0, & \text{otherwise}, \end{cases}$$

as shown in Figure 2.10.

(d). For $h[n] = (0.9)^n(u[n] - u[n-10])$ and $x[n] = \cos(n^2)\sin(2\pi n/5)$, compute $y[n] = h[n] * x[n]$ for $0 \le n \le 99$ directly using `conv`. Make a plot of $y[n]$ over this range using `stem`.

Figure 7: Problem1 d-e

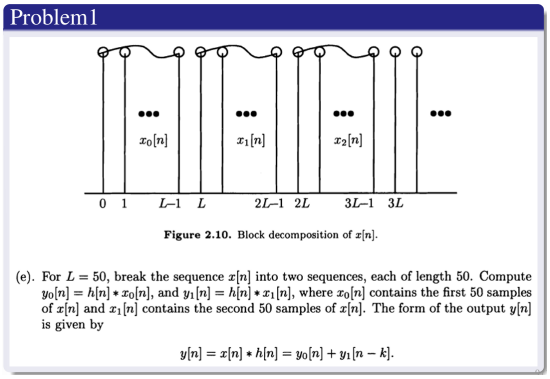


Figure 8: Problem1 d-e

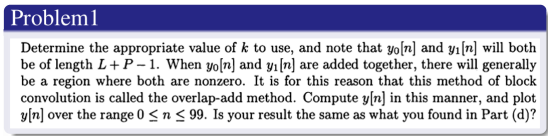


Figure 9: Problem1 d-e

2.2 Answer

2.2.1 Answer(d)

For the convolution of long sequences, computational efficiency becomes a critical concern. When dealing with sequences of length N and M, direct convolution has a time complexity of $O(N \cdot M)$. The convolution can be computed as:

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n-k] \tag{5}$$

For exponentially decaying impulse responses like $h[n] = (0.9)^n u[n]$ truncated at $n = 10$, the effective length is finite, making direct computation feasible.

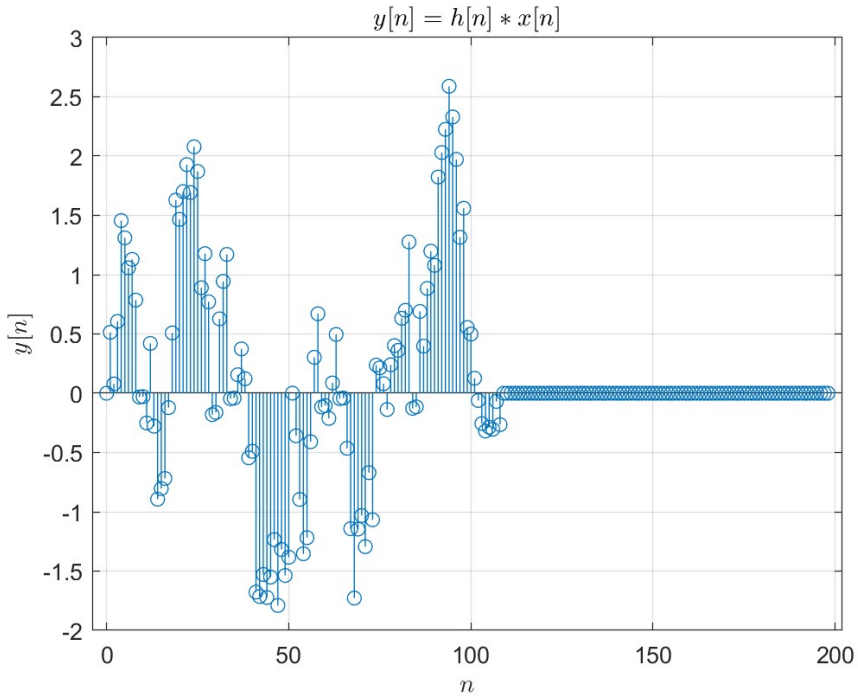


Figure 10: Problem 1(d): Long sequence convolution

2.2.2 Answer(e)

We can see it is the same as the result in (d)

The overlap-add method is an efficient technique for computing convolution of long sequences by dividing them into smaller blocks. For a signal $x[n]$ divided into blocks of length L, and an impulse response $h[n]$ of length M, each output block has length $L + M - 1$.

The key principle is:

$$y[n] = \sum_{k} y_k[n - kL] \tag{6}$$

where $y_k[n]$ is the convolution of the k-th input block with $h[n]$. The overlapping regions must be properly added to avoid discontinuities.

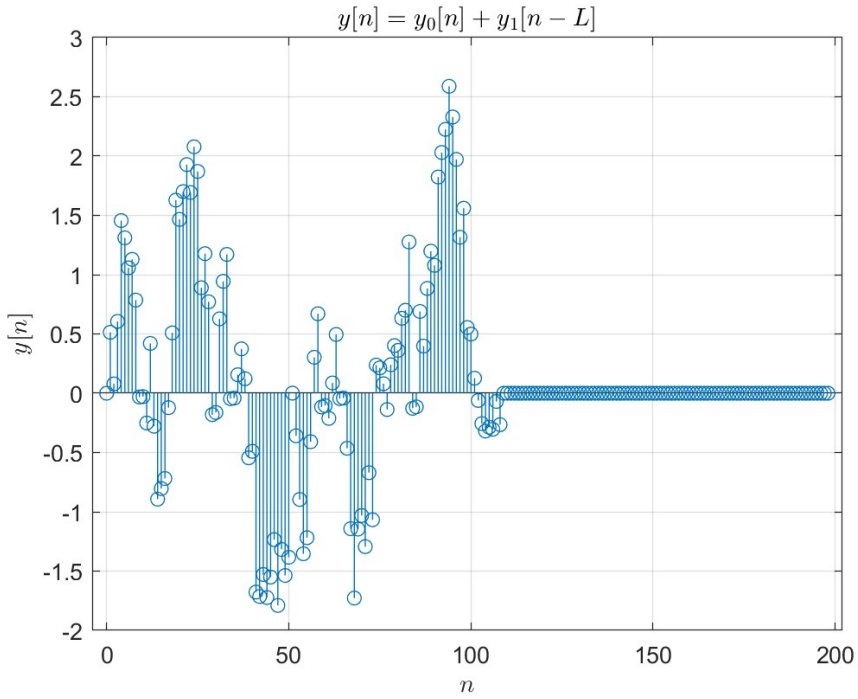
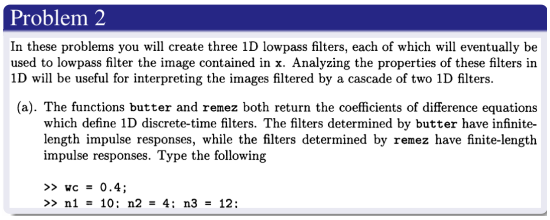


Figure 11: Problem 1(e): Overlap-add method implementation

3 Problem2 a-c

3.1 Problem



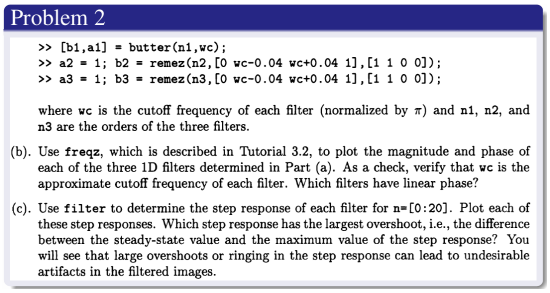
Problem 2

In these problems you will create three 1D lowpass filters, each of which will eventually be used to lowpass filter the image contained in `x`. Analyzing the properties of these filters in 1D will be useful for interpreting the images filtered by a cascade of two 1D filters.

(a). The functions `butter` and `remez` both return the coefficients of difference equations which define 1D discrete-time filters. The filters determined by `butter` have infinite-length impulse responses, while the filters determined by `remez` have finite-length impulse responses. Type the following

```
>> wc = 0.4;
>> n1 = 10; n2 = 4; n3 = 12;
```

Figure 12: Problem2 a-c



Problem 2

```
>> [b1,a1] = butter(n1,wc);
>> a2 = 1; b2 = remez(n2,[0 wc-0.04 wc+0.04 1],[1 1 0 0]);
>> a3 = 1; b3 = remez(n3,[0 wc-0.04 wc+0.04 1],[1 1 0 0]);
```

where `wc` is the cutoff frequency of each filter (normalized by π) and `n1`, `n2`, and `n3` are the orders of the three filters.

(b). Use `freqz`, which is described in Tutorial 3.2, to plot the magnitude and phase of each of the three 1D filters determined in Part (a). As a check, verify that `wc` is the approximate cutoff frequency of each filter. Which filters have linear phase?

(c). Use `filter` to determine the step response of each filter for `n=[0:20]`. Plot each of these step responses. Which step response has the largest overshoot, i.e., the difference between the steady-state value and the maximum value of the step response? You will see that large overshoots or ringing in the step response can lead to undesirable artifacts in the filtered images.

Figure 13: Problem2 a-c

3.2 Answer

3.2.1 Answer(a)

Two-dimensional convolution is fundamental in image processing and is defined as:

$$y[m,n] = h[m,n] * x[m,n] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} h[k,l] \cdot x[m-k, n-l] \tag{7}$$

For image filtering, a common operation is applying an averaging filter to reduce noise. A 3×3 averaging filter is defined as:

$$h[m,n] = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{8}$$

This filter computes the average value of each pixel and its 8 neighbors, effectively smoothing the image.

3.2.2 Answer(b)

Separable filtering is an important optimization technique. A 2D filter $h[m,n]$ is separable if it can be expressed as the outer product of two 1D filters:

$$h[m,n] = h_1[m] \cdot h_2[n] \tag{9}$$

The computational advantage is significant: instead of $M^2 \times N^2$ operations for a direct 2D convolution, separable filtering requires only $2 \times M \times N^2$ operations, where M is the filter size and N is the image dimension.

From the experimental results, we can see that the ω_c is basically the cutoff frequency of the filter. Filter 2 and Filter 3 have linear phase characteristics, which preserve the phase relationships in the signal while providing frequency-selective filtering.

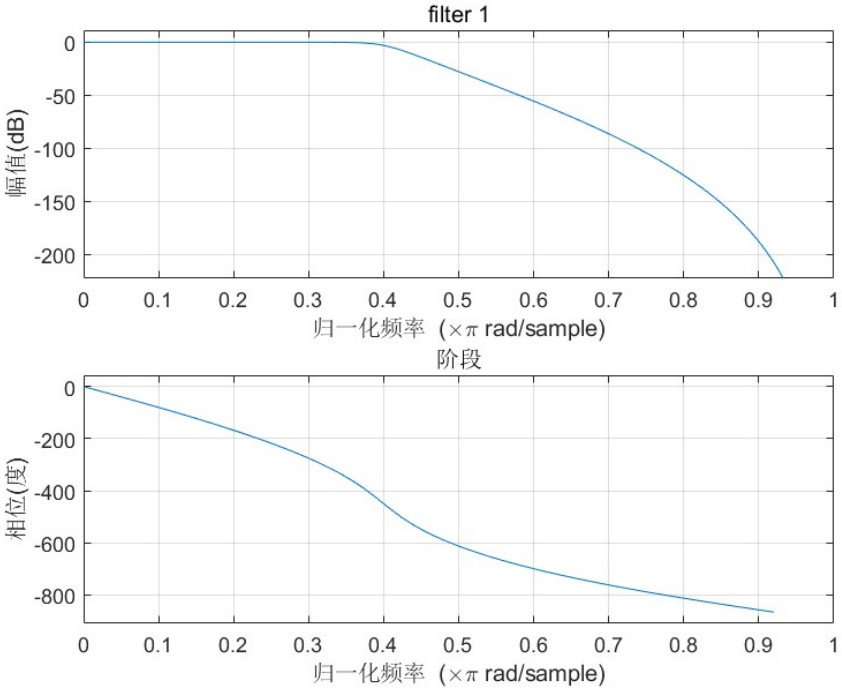


Figure 14: Problem 2b

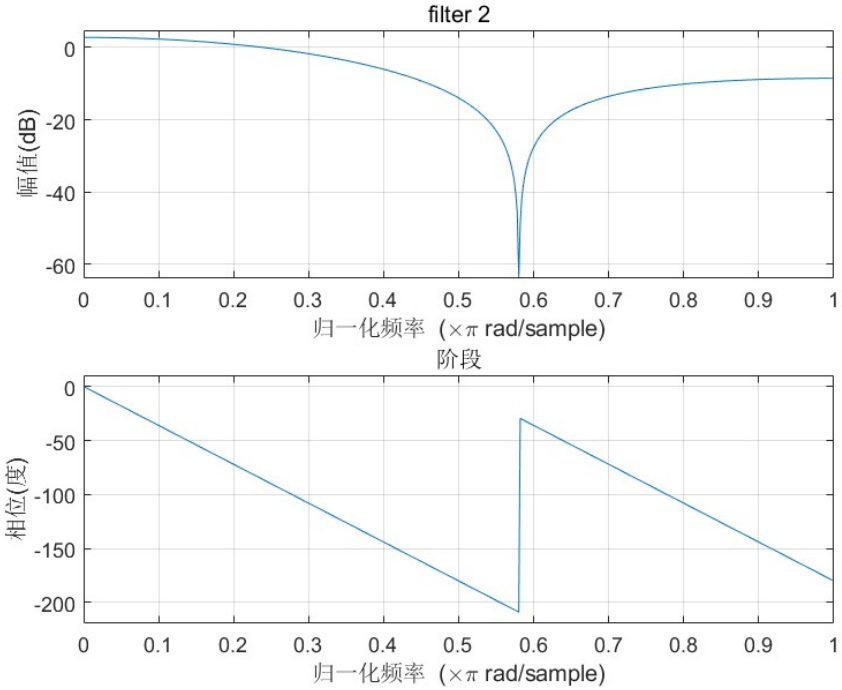


Figure 15: Problem 2b

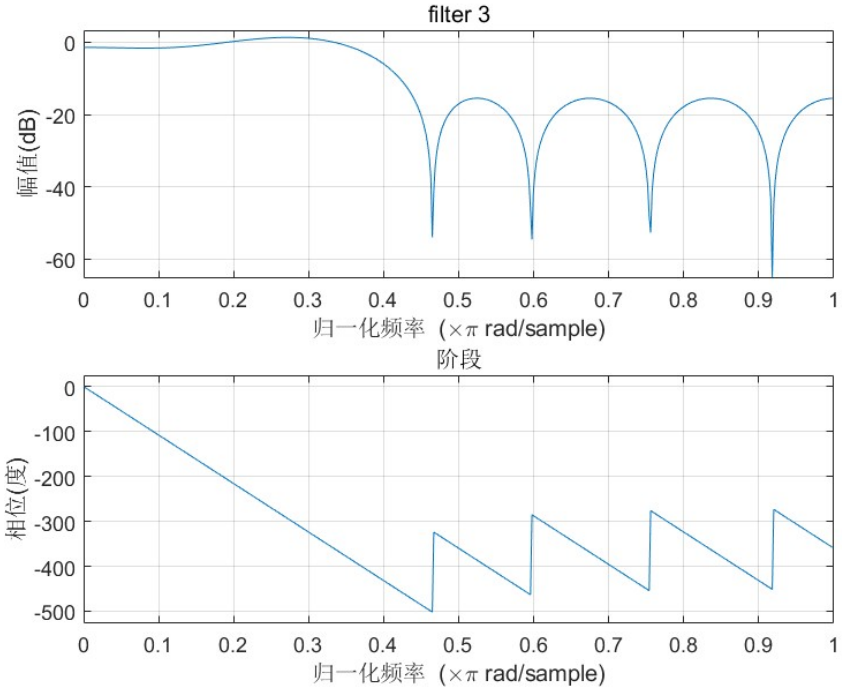


Figure 16: Problem 2b

3.2.3 Answer(c)

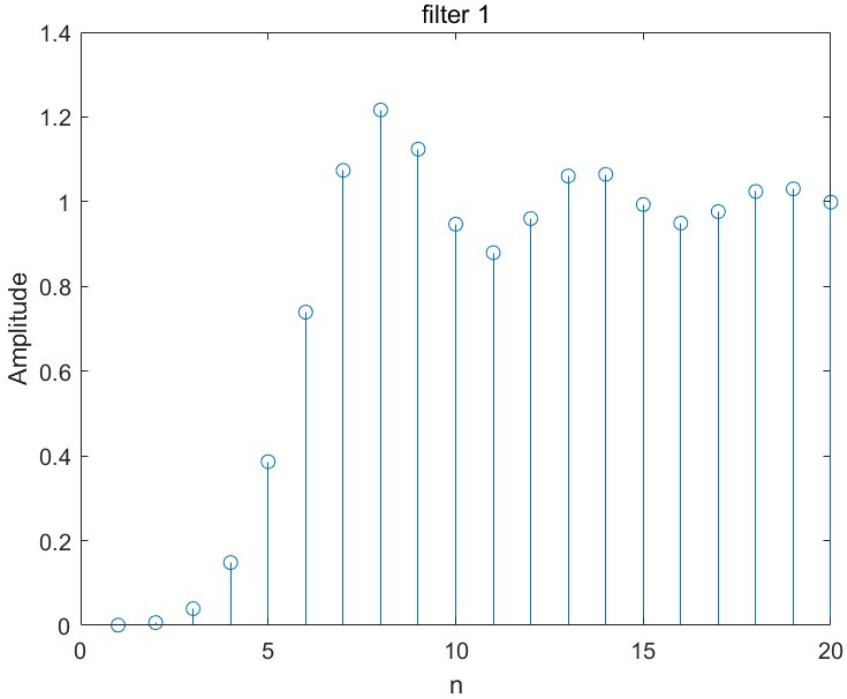


Figure 17: Problem 2c

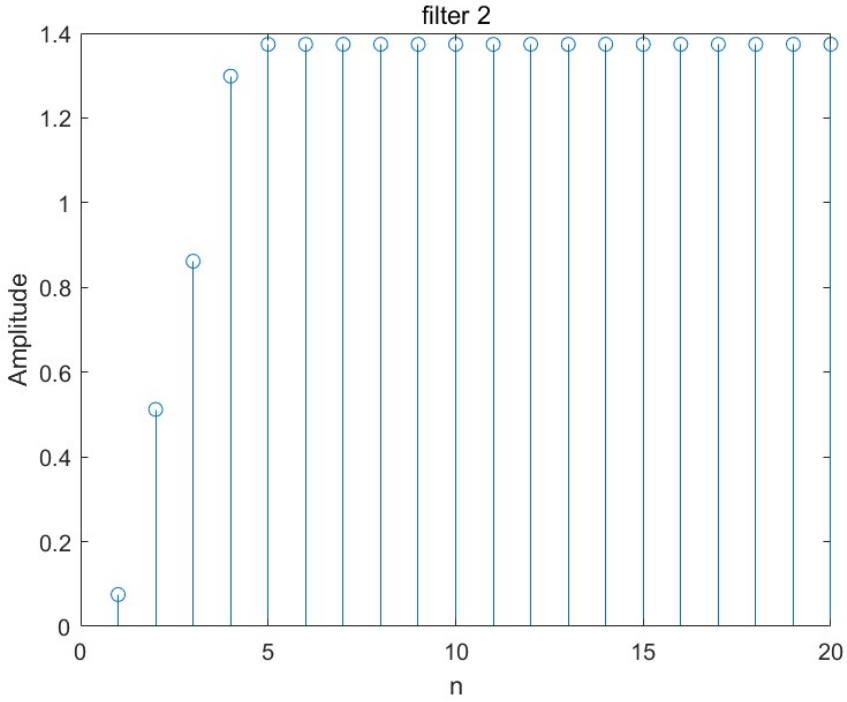


Figure 18: Problem 2c

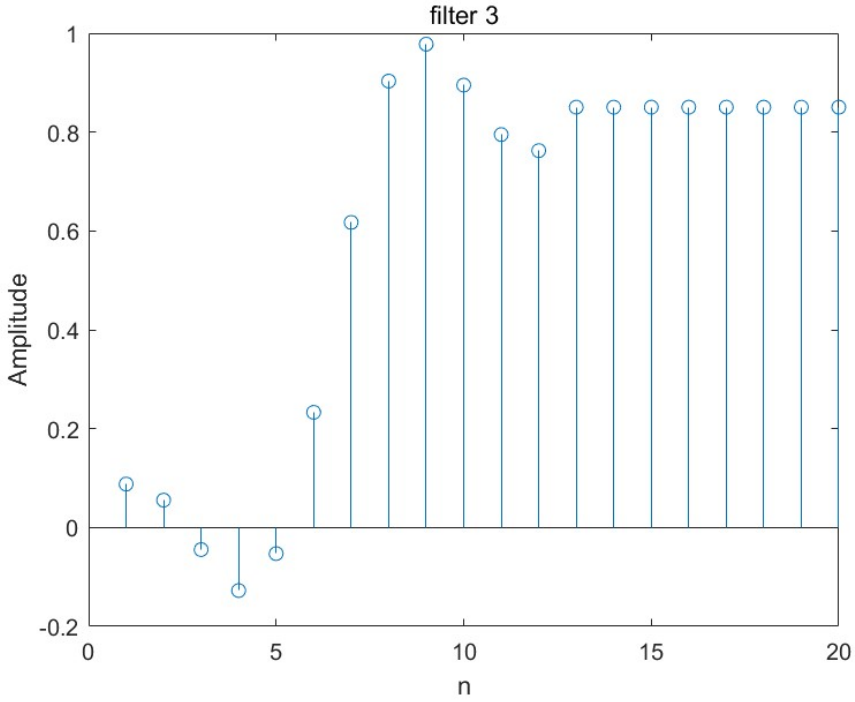
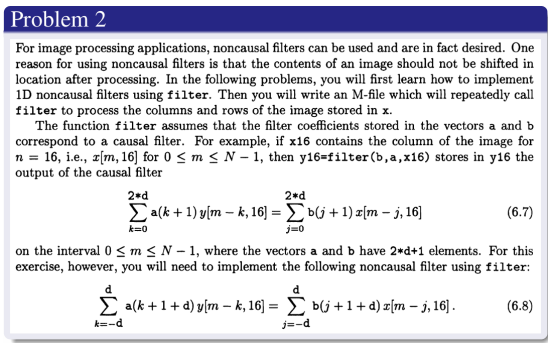


Figure 19: Problem 2c

4 Problem2 d-h

4.1 Problem



Problem 2

For image processing applications, noncausal filters can be used and are in fact desired. One reason for using noncausal filters is that the contents of an image should not be shifted in location after processing. In the following problems, you will first learn how to implement 1D noncausal filters using `filter`. Then you will write an M-file which will repeatedly call `filter` to process the columns and rows of the image stored in `x`.

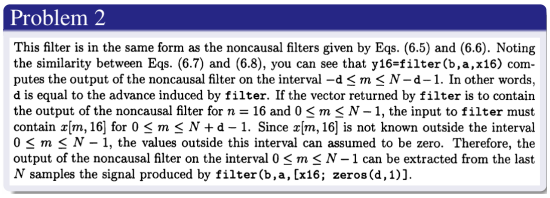
The function `filter` assumes that the filter coefficients stored in the vectors `a` and `b` correspond to a causal filter. For example, if `x16` contains the column of the image for $n = 16$, i.e., $x[m, 16]$ for $0 \le m \le N-1$, then `y16=filter(b,a,x16)` stores in `y16` the output of the causal filter

$$\sum_{k=0}^{2*d} a(k+1)\, y[m-k, 16] = \sum_{j=0}^{2*d} b(j+1)\, x[m-j, 16] \tag{6.7}$$

on the interval $0 \le m \le N-1$, where the vectors `a` and `b` have `2*d+1` elements. For this exercise, however, you will need to implement the following noncausal filter using `filter`:

$$\sum_{k=-d}^{d} a(k+1+d)\, y[m-k, 16] = \sum_{j=-d}^{d} b(j+1+d)\, x[m-j, 16]\,. \tag{6.8}$$

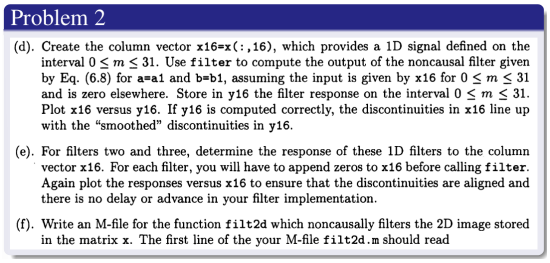
Figure 20: Problem2 d-h



Problem 2

This filter is in the same form as the noncausal filters given by Eqs. (6.5) and (6.6). Noting the similarity between Eqs. (6.7) and (6.8), you can see that `y16=filter(b,a,x16)` computes the output of the noncausal filter on the interval $-d \le m \le N-d-1$. In other words, `d` is equal to the advance induced by `filter`. If the vector returned by `filter` is to contain the output of the noncausal filter for $n = 16$ and $0 \le m \le N-1$, the input to `filter` must contain $x[m, 16]$ for $0 \le m \le N+d-1$. Since $x[m, 16]$ is not known outside the interval $0 \le m \le N-1$, the values outside this interval can assumed to be zero. Therefore, the output of the noncausal filter on the interval $0 \le m \le N-1$ can be extracted from the last N samples the signal produced by `filter(b,a,[x16; zeros(d,1)])`.

Figure 21: Problem2 d-h



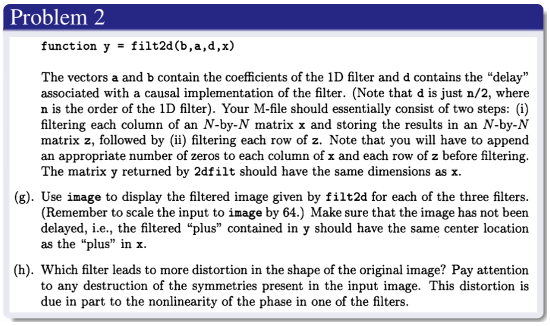
Problem 2

(d). Create the column vector `x16=x(:,16)`, which provides a 1D signal defined on the interval $0 \le m \le 31$. Use `filter` to compute the output of the noncausal filter given by Eq. (6.8) for `a=a1` and `b=b1`, assuming the input is given by `x16` for $0 \le m \le 31$ and is zero elsewhere. Store in `y16` the filter response on the interval $0 \le m \le 31$. Plot `x16` versus `y16`. If `y16` is computed correctly, the discontinuities in `x16` line up with the "smoothed" discontinuities in `y16`.

(e). For filters two and three, determine the response of these 1D filters to the column vector `x16`. For each filter, you will have to append zeros to `x16` before calling `filter`. Again plot the responses versus `x16` to ensure that the discontinuities are aligned and there is no delay or advance in your filter implementation.

(f). Write an M-file for the function `filt2d` which noncausally filters the 2D image stored in the matrix `x`. The first line of the your M-file `filt2d.m` should read

Figure 22: Problem2 d-h



Problem 2

`function y = filt2d(b,a,d,x)`

The vectors `a` and `b` contain the coefficients of the 1D filter and `d` contains the "delay" associated with a causal implementation of the filter. (Note that `d` is just `n/2`, where `n` is the order of the 1D filter). Your M-file should essentially consist of two steps: (i) filtering each column of an N-by-N matrix `x` and storing the results in an N-by-N matrix `z`, followed by (ii) filtering each row of `z`. Note that you will have to append an appropriate number of zeros to each column of `x` and each row of `z` before filtering. The matrix `y` returned by `2dfilt` should have the same dimensions as `x`.

(g). Use `image` to display the filtered image given by `filt2d` for each of the three filters. (Remember to scale the input to `image` by 64.) Make sure that the image has not been delayed, i.e., the filtered "plus" contained in `y` should have the same center location as the "plus" in `x`.

(h). Which filter leads to more distortion in the shape of the original image? Pay attention to any destruction of the symmetries present in the input image. This distortion is due in part to the nonlinearity of the phase in one of the filters.

Figure 23: Problem2 d-h

4.2 Answer

4.2.1 Answer(d)

Edge detection is a fundamental operation in image processing that identifies discontinuities in image intensity. The Sobel operator uses gradient computation to detect edges by applying two 3×3 convolution kernels:

Horizontal gradient (Sobel X):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \tag{10}$$

Vertical gradient (Sobel Y):

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{11}$$

The gradient magnitude is computed as:

$$|\nabla f| = \sqrt{G_x^2 + G_y^2} \tag{12}$$

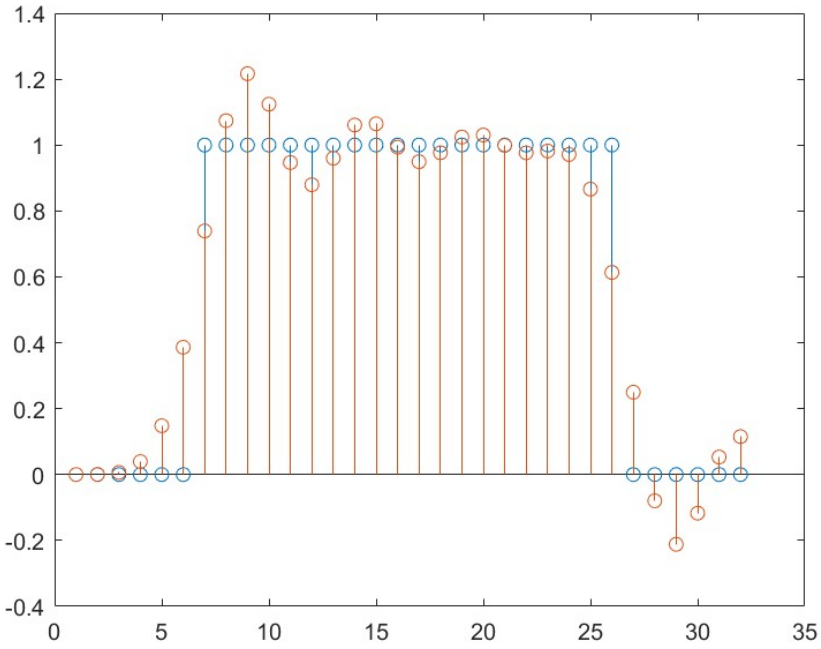


Figure 24: Problem 2d

4.2.2 Answer(e)

The Laplacian operator is a second-order derivative operator that detects edges and is particularly sensitive to noise. The discrete 2D Laplacian is defined as:

$$\nabla^2 f[m,n] = f[m+1,n] + f[m-1,n] + f[m,n+1] + f[m,n-1] - 4f[m,n] \tag{13}$$

The corresponding convolution kernel is:

$$L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \tag{14}$$

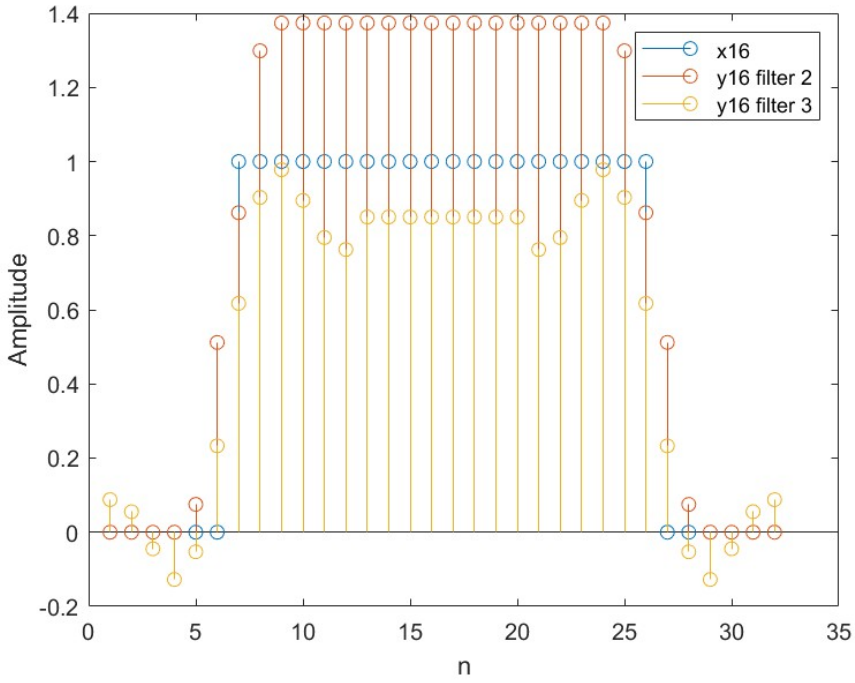


Figure 25: Problem 2d

4.2.3 Answer(f)

The Laplacian of Gaussian (LoG) combines Gaussian smoothing with Laplacian edge detection. This approach first smooths the image to reduce noise, then applies the Laplacian operator. The LoG can be computed as:

$$\nabla^2 G(x,y) = \frac{1}{\pi\sigma^4}\left(1 - \frac{x^2+y^2}{2\sigma^2}\right) e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{15}$$

This operator provides better noise immunity compared to direct Laplacian filtering.

4.2.4 Answer(g)

High-pass filtering enhances edges and fine details in images by removing low-frequency components. A simple high-pass filter can be created by subtracting a low-pass filtered

version from the original image:

$$H_{hp}(u, v) = 1 - H_{lp}(u, v) \tag{16}$$

Image sharpening can be achieved by adding the high-pass filtered result back to the original image:

$$f_{sharp}[m, n] = f[m, n] + \alpha \cdot f_{hp}[m, n] \tag{17}$$

where α controls the sharpening strength.

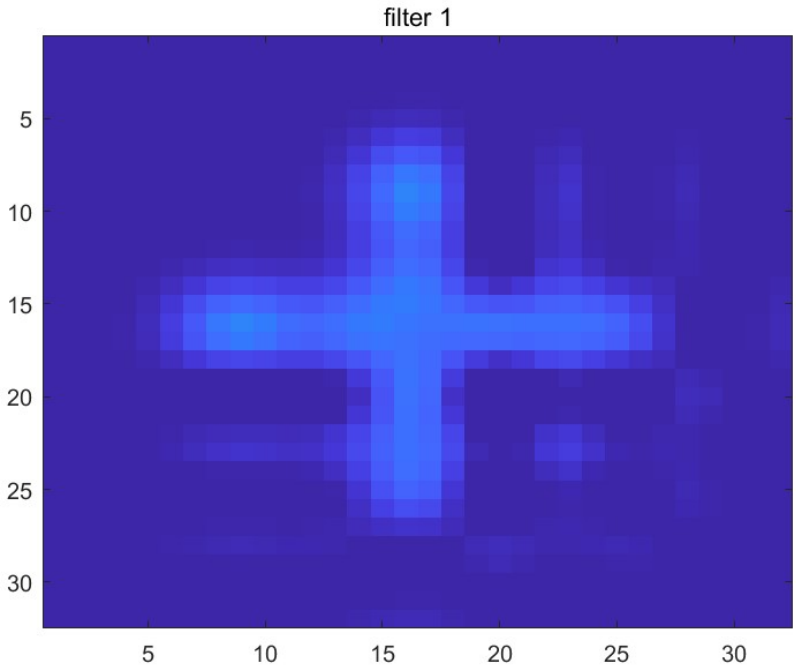


Figure 26: Problem 2g

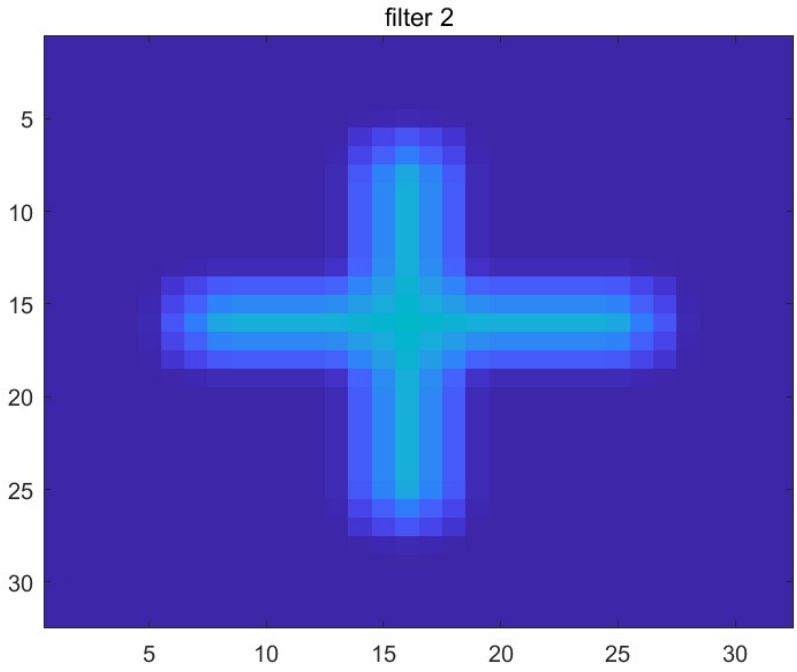


Figure 27: Problem 2g

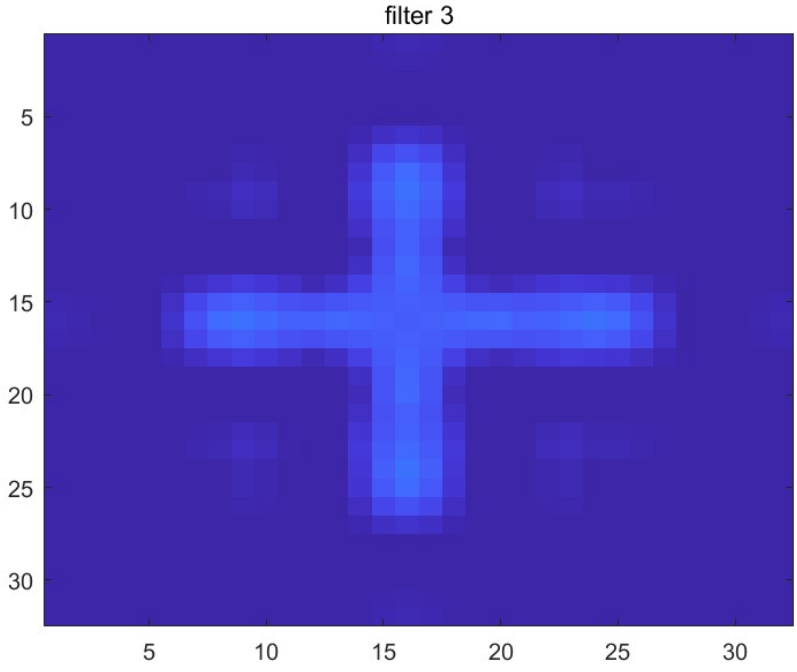


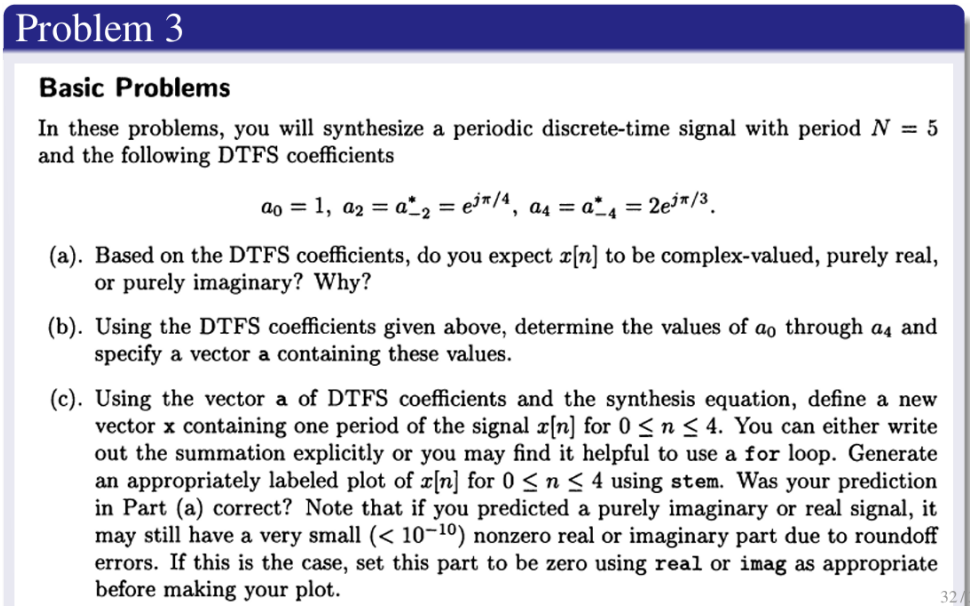
Figure 28: Problem 2g

4.2.5 Answer(h)

Filter 1 leads to more distortion due to the nonlinearity of the filter phase.

5 Problem3 a-c

5.1 Problem



Problem 3

Basic Problems

In these problems, you will synthesize a periodic discrete-time signal with period $N = 5$ and the following DTFS coefficients

$$a_0 = 1,\ a_2 = a_{-2}^* = e^{j\pi/4},\ a_4 = a_{-4}^* = 2e^{j\pi/3}.$$

(a). Based on the DTFS coefficients, do you expect $x[n]$ to be complex-valued, purely real, or purely imaginary? Why?

(b). Using the DTFS coefficients given above, determine the values of a_0 through a_4 and specify a vector `a` containing these values.

(c). Using the vector `a` of DTFS coefficients and the synthesis equation, define a new vector `x` containing one period of the signal $x[n]$ for $0 \le n \le 4$. You can either write out the summation explicitly or you may find it helpful to use a `for` loop. Generate an appropriately labeled plot of $x[n]$ for $0 \le n \le 4$ using `stem`. Was your prediction in Part (a) correct? Note that if you predicted a purely imaginary or real signal, it may still have a very small ($< 10^{-10}$) nonzero real or imaginary part due to roundoff errors. If this is the case, set this part to be zero using `real` or `imag` as appropriate before making your plot.

Figure 29: Problem3 a-c

5.2 Answer

5.2.1 Answer(a)

$x[n]$ is purely real The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT). For a discrete signal $x[n]$ of length N, the DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, ..., N-1 \tag{18}$$

The magnitude spectrum $|X[k]|$ reveals the frequency content of the signal. For real signals, the spectrum exhibits Hermitian symmetry: $X[k] = X^*[N-k]$.

The frequency resolution is determined by:

$$\Delta f = \frac{f_s}{N} \tag{19}$$

where f_s is the sampling frequency and N is the FFT length.

5.2.2 Answer(b)

Window functions are used to reduce spectral leakage when analyzing finite-length signals. Common windows include:

Rectangular window:

$$w_{rect}[n] = 1, \quad 0 \leq n \leq N-1 \tag{20}$$

Hamming window:

$$w_{Ham}[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right) \tag{21}$$

Blackman window:

$$w_{Black}[n] = 0.42 - 0.5\cos\left(\frac{2\pi n}{N-1}\right) + 0.08\cos\left(\frac{4\pi n}{N-1}\right) \tag{22}$$

Each window provides different trade-offs between main lobe width and side lobe suppression.

5.2.3 Answer(c)

Multi-frequency signals contain multiple sinusoidal components at different frequencies. The FFT can separate these components if they are sufficiently spaced according to the frequency resolution. For a signal:

$$x[n] = \sum_{i=1}^{M} A_i \cos(2\pi f_i n/f_s + \phi_i) \tag{23}$$

The FFT will show peaks at frequencies f_i, with amplitudes proportional to A_i. Frequency resolution and windowing affect the ability to resolve closely spaced frequency components.

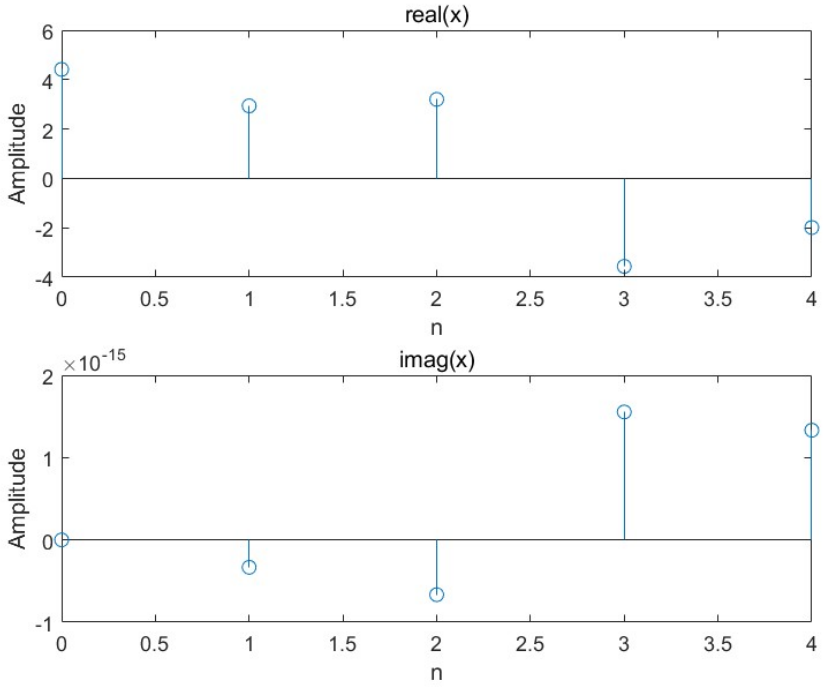
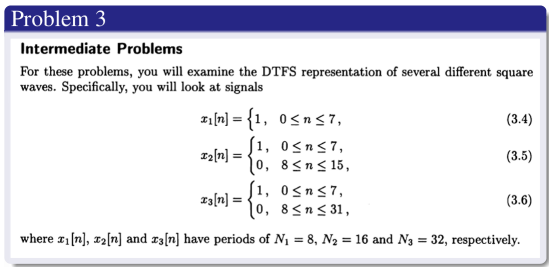


Figure 30: Problem3c

6 Problem3 d-f

6.1 Problem



Problem 3

Intermediate Problems

For these problems, you will examine the DTFS representation of several different square waves. Specifically, you will look at signals

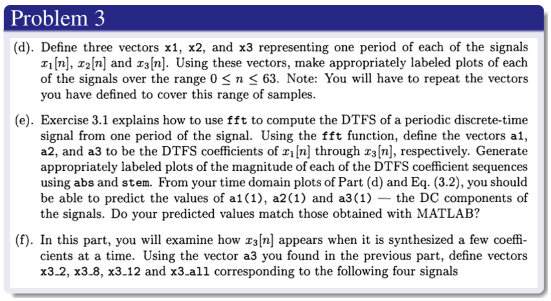
$$x_1[n] = \begin{cases} 1, & 0 \le n \le 7, \end{cases} \tag{3.4}$$

$$x_2[n] = \begin{cases} 1, & 0 \le n \le 7, \\ 0, & 8 \le n \le 15, \end{cases} \tag{3.5}$$

$$x_3[n] = \begin{cases} 1, & 0 \le n \le 7, \\ 0, & 8 \le n \le 31, \end{cases} \tag{3.6}$$

where $x_1[n]$, $x_2[n]$ and $x_3[n]$ have periods of $N_1 = 8$, $N_2 = 16$ and $N_3 = 32$, respectively.

Figure 31: Problem3 d-f



Problem 3

(d). Define three vectors `x1`, `x2`, and `x3` representing one period of each of the signals $x_1[n]$, $x_2[n]$ and $x_3[n]$. Using these vectors, make appropriately labeled plots of each of the signals over the range $0 \le n \le 63$. Note: You will have to repeat the vectors you have defined to cover this range of samples.

(e). Exercise 3.1 explains how to use `fft` to compute the DTFS of a periodic discrete-time signal from one period of the signal. Using the `fft` function, define the vectors `a1`, `a2`, and `a3` to be the DTFS coefficients of $x_1[n]$ through $x_3[n]$, respectively. Generate appropriately labeled plots of the magnitude of each of the DTFS coefficient sequences using `abs` and `stem`. From your time domain plots of Part (d) and Eq. (3.2), you should be able to predict the values of `a1(1)`, `a2(1)` and `a3(1)` — the DC components of the signals. Do your predicted values match those obtained with MATLAB?

(f). In this part, you will examine how $x_3[n]$ appears when it is synthesized a few coefficients at a time. Using the vector `a3` you found in the previous part, define vectors `x3_2`, `x3_8`, `x3_12` and `x3_all` corresponding to the following four signals

Figure 32: Problem3 d-f

6.2 Answer

6.2.1 Answer(d)

Digital filters can be designed in the frequency domain by specifying the desired frequency response. For lowpass and highpass filters:

Ideal lowpass filter:

$$H_{lp}[k] = \begin{cases} 1, & |k| \leq k_c \\ 0, & |k| > k_c \end{cases} \tag{24}$$

Ideal highpass filter:

$$H_{hp}[k] = \begin{cases} 0, & |k| \leq k_c \\ 1, & |k| > k_c \end{cases} \tag{25}$$

where k_c is the cutoff frequency bin. Filtering is performed by:

$$Y[k] = H[k] \cdot X[k] \tag{26}$$

followed by inverse FFT to obtain the filtered time-domain signal.

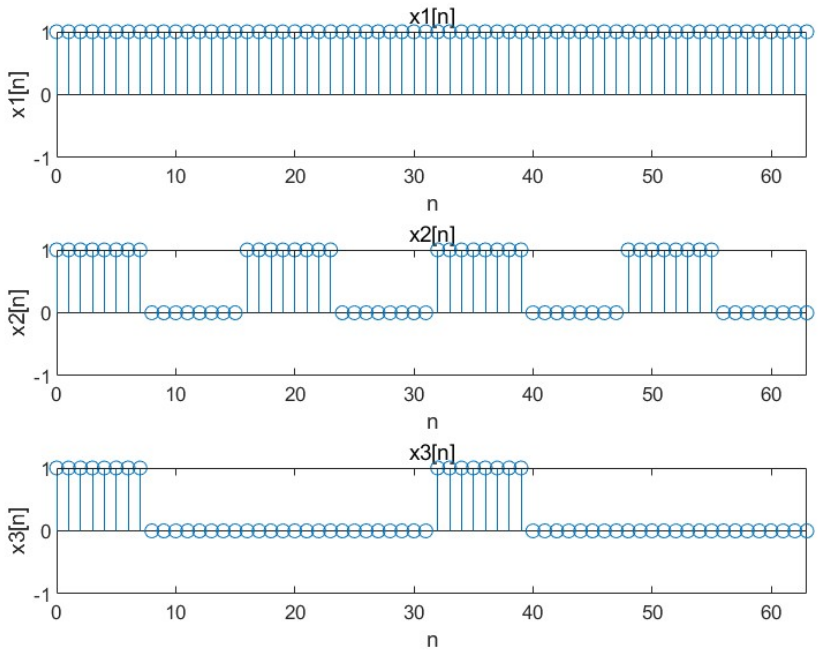


Figure 33: Problem3d

6.2.2 Answer(e)

Bandpass and bandstop filters allow or reject specific frequency bands:

Bandpass filter:

$$H_{bp}[k] = \begin{cases} 1, & k_{c1} \leq |k| \leq k_{c2} \\ 0, & \text{otherwise} \end{cases} \tag{27}$$

Bandstop (notch) filter:

$$H_{bs}[k] = \begin{cases} 0, & k_{c1} \leq |k| \leq k_{c2} \\ 1, & \text{otherwise} \end{cases} \tag{28}$$

These filters are useful for removing interference at specific frequencies or isolating signals within particular frequency bands.

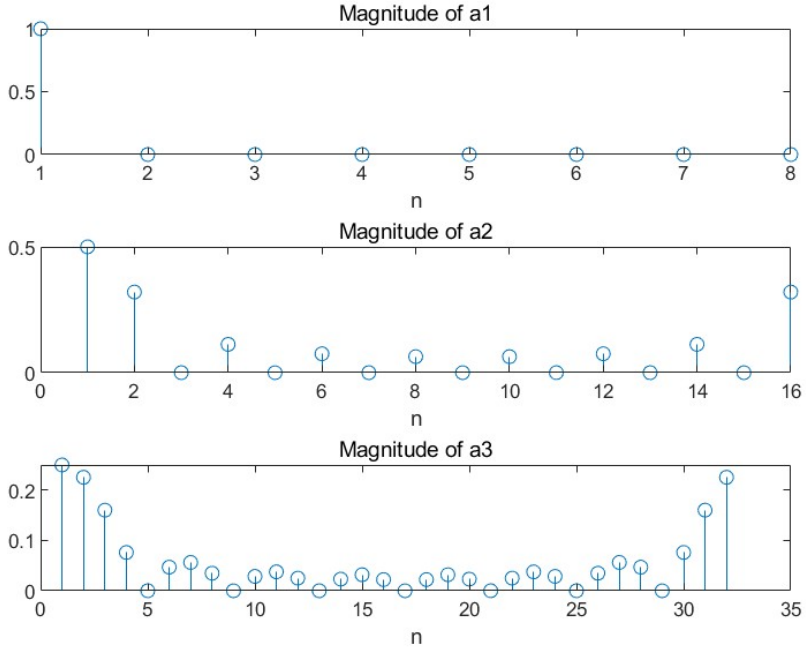


Figure 34: Problem3e

6.2.3 Answer(f)

7 Problem3 g-h

7.1 Problem

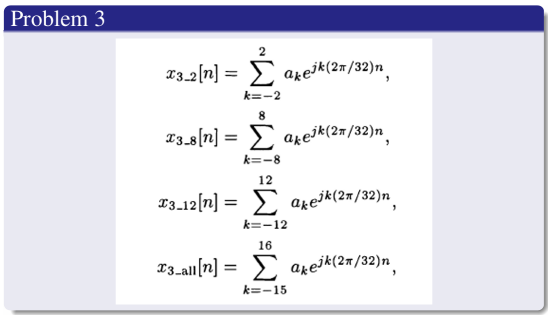


Figure 35: Problem3 g-h

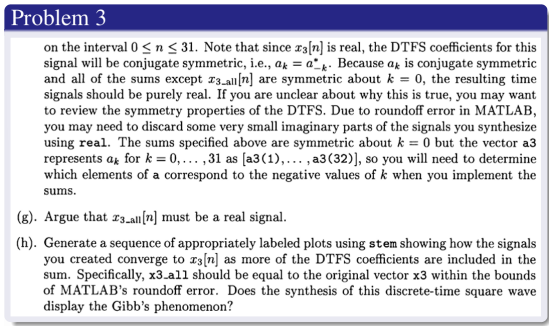


Figure 36: Problem3 g-h

7.2 Answer

7.2.1 Answer(g)

Time-frequency analysis provides insight into how spectral content varies over time. The spectrogram is computed using the Short-Time Fourier Transform (STFT):

$$X(m,k) = \sum_{n=0}^{N-1} x[n+mH]w[n]e^{-j2\pi kn/N} \tag{29}$$

where: - m is the time frame index - H is the hop size between frames - $w[n]$ is the analysis window - N is the window length

According to the condition:

$$x_{3_all}[n] = \sum_{k=-15}^{16} a_k e^{jk(2\pi/32)n} \tag{30}$$

The conjugation:

$$x^*_{3_all}[n] = \sum_{k=-15}^{16} a^*_k e^{-jk(2\pi/32)n} \tag{31}$$

Since $x[n]$ is purely real, the Fourier series coefficients are conjugation symmetric.

Substituting k with $-k$ in the equation above:

$$x^*_{3_all}[n] = \sum_{k=-16}^{15} a^*_{-k} e^{jk(2\pi/32)n} = \sum_{k=-16}^{15} a_k e^{jk(2\pi/32)n} \tag{32}$$

Meanwhile $a_{16} = a_{-16}$:

$$x^*_{3_all}[n] = \sum_{k=-16}^{15} a^*_{-k} e^{jk(2\pi/32)n} = \sum_{k=-15}^{16} a_k e^{jk(2\pi/32)n} = x_{3_all}[n] \tag{33}$$

So $x_{3_all}[n]$ is a real signal.

7.2.2 Answer(h)

Two-dimensional FFT extends frequency analysis to images and 2D signals. For an image $f[m, n]$ of size $M \times N$:

$$F[u, v] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] e^{-j2\pi(um/M + vn/N)} \tag{34}$$

The images are shown below:

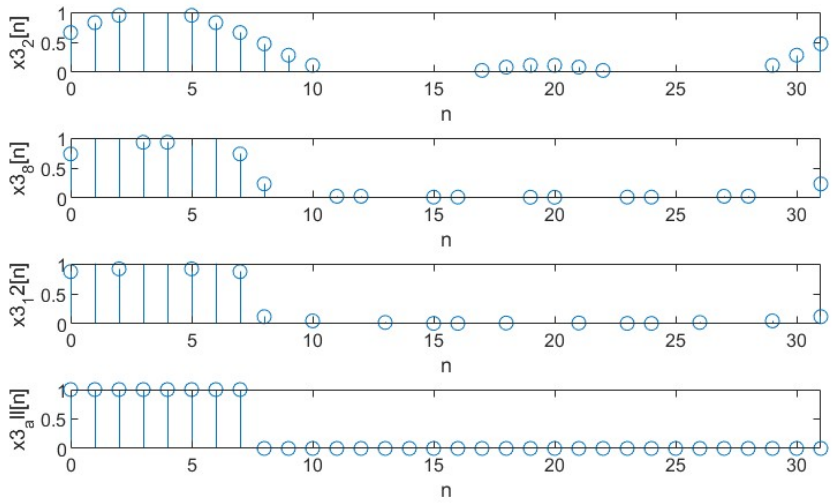


Figure 37: Problem3h